Draw a line!

1 MAKE IT!

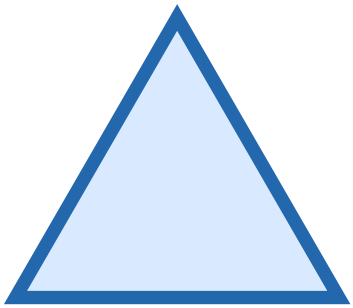
```
my_line = [
    ("up", -3, 0),
    ("heading", 0),
    ("color", "blue"),
    ("width", 4),
    ("down",),
    ("forward", 6),
]
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Blue Line", my_line),
]
```

Copy the code into student_images.py

Triangle!



1 MAKE IT!

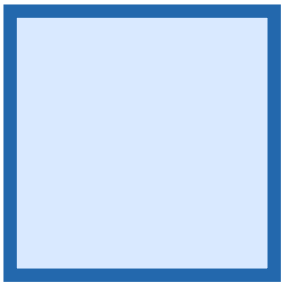
```
my_triangle = triangle(x=-2, y=-2,
                       size=3, angle=45,
                       color="blue", fill="skyblue",
                       speed=5)
```

2 ADD IT TO YOUR SEQUENCE!

```
# Add triangle to your sequence
image_sequence = [
    ("Triangle", my_triangle),
]
```

Copy the code into student_images.py

Square!



1 MAKE IT!

```
my_square = square(x=-1, y=-1, size=2,
                   angle=20, color="green",
                   fill="lightgreen")
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Square", my_square),
]
```

Copy the code into student_images.py

Rectangle!



1 MAKE IT!

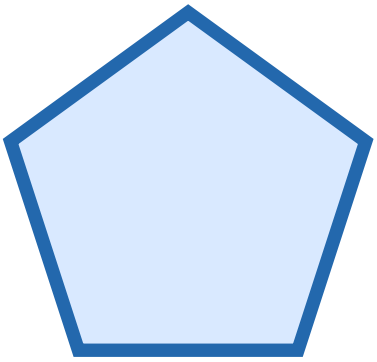
```
my_rectangle = rectangle(x=-3, y=-2,
                         width=5, height=2,
                         angle=0, color="blue",
                         fill="lightblue")
```

2 ADD IT TO YOUR SEQUENCE!

```
# Add rectangle to your sequence
image_sequence = [
    ("Rectangle", my_rectangle),
]
```

Copy the code into student_images.py

Pentagon!



1 MAKE IT!

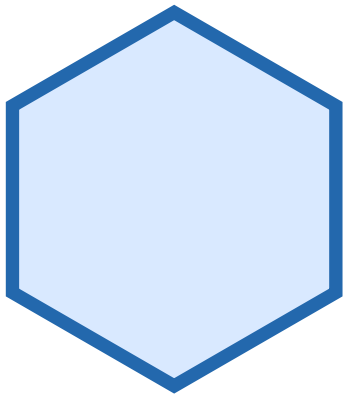
```
my_pentagon = pentagon(x=-1, y=-2,
                       size=2, angle=0,
                       color="orange", fill="yellow")
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Pentagon", my_pentagon),
]
```

Copy the code into student_images.py

Hexagon!



1 MAKE IT!

```
my_hexagon = hexagon(x=-1, y=-2,
                     size=2, color="purple",
                     fill="lavender")
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Hexagon", my_hexagon),
]
```

Copy the code into student_images.py

Star!



1 MAKE IT!

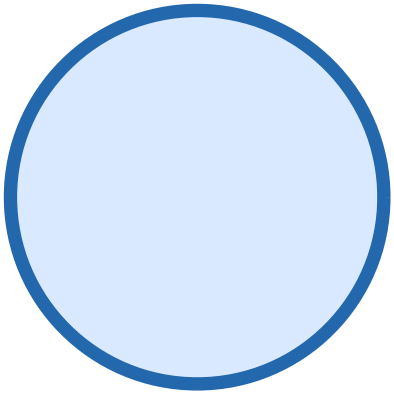
```
my_star = star(x=-2, y=0, size=3,
               angle=0, color="purple",
               fill="yellow", speed=5)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Star", my_star),
]
```

Copy the code into student_images.py

Circle!



1 MAKE IT!

```
my_circle = circle(x=0, y=0, radius=1,
                   color="purple", fill="pink",
                   speed=5)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Circle", my_circle),
]
```

Copy the code into student_images.py

Spiral!



1 MAKE IT!

```
my_spiral = spiral(x=0, y=0,
                   turn=90, steps=12,
                   growth=0.2, color="purple",
                   speed=5)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Spiral", my_spiral),
]
```

Copy the code into student_images.py

Letter a!



1 MAKE IT!

```
my_a = letter_a(x=-3, y=-2, size=3,
                color="red", speed=5)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Letter A", my_a),
]
```

Copy the code into student_images.py

Letter h!



1 MAKE IT!

```
my_h = letter_h(x=-2, y=-2, size=3,
                color="purple", speed=5)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Letter H", my_h),
]
```

Copy the code into student_images.py

Curved letter s!



1 MAKE IT!

```
my_s = letter_s(x=0, y=2, radius=1,
                color="purple", speed=5)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Letter S", my_s),
]
```

Copy the code into student_images.py

Venetian blinds!



1 MAKE IT!

```
my_blinds = venetian_blind(
    x=-4, y=-3, width=8,
    slats=7, slat_height=0.55,
    gap=0.3, color="steelblue",
    fill="lightblue", speed=5
)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Venetian Blinds", my_blinds),
]
```

Copy the code into student_images.py

Zigzag!



1 MAKE IT!

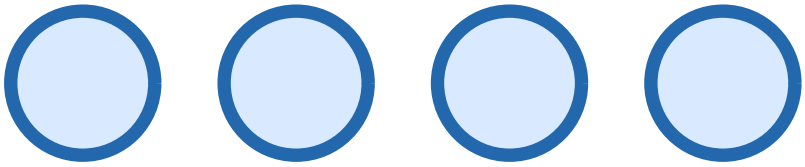
```
my_zigzag = zigzag(x=-4, y=0,
                   steps=6, segment=1,
                   angle=45, color="orange")
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Zigzag", my_zigzag),
]
```

Copy the code into student_images.py

Repeat a shape!



1 MAKE IT!

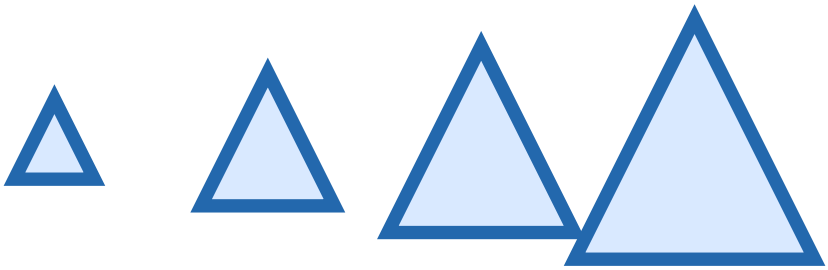
```
my_row = repeat_shape(
    circle, count=4,
    x=-4, y=0, dx=2, dy=0,
    radius=0.4, color="blue"
)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Repeated Shapes", my_row),
]
```

Copy the code into student_images.py

Growing shapes!



1 MAKE IT!

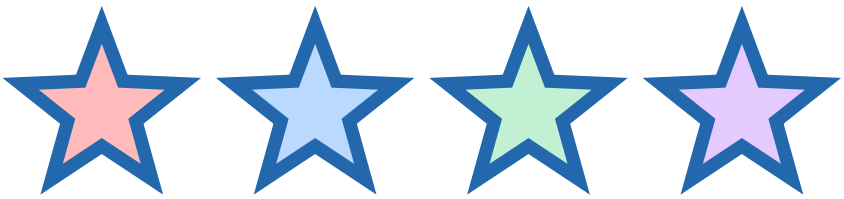
```
my_growth = growing_shapes(
    triangle, count=4,
    x=-3, y=-3,
    start_size=1, step=0.5,
    color="green"
)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Growing Shapes", my_growth),
]
```

Copy the code into student_images.py

Use random colours!



1 MAKE IT!

```
my_colours = random_colors(
    star, count=4,
    x=-4, y=0, dx=2,
    size=1,
    palette=["red", "blue", "green", "purple"]
)
```

2 ADD IT TO YOUR SEQUENCE!

```
image_sequence = [
    ("Random Colours", my_colours),
]
```

Copy the code into student_images.py

Choose a background!

1 MAKE IT!

```
# Put this near the top of student_images.py
background = "beach"
# Other choices:
# "space", "forest", "city", "classroom", "none"
# Or add a PNG/GIF to backgrounds/:
# background = "my_picture.png"
```

2 ADD IT TO YOUR SEQUENCE!

```
# Backgrounds are not added to image_sequence.
sun = circle(x=2, y=2, radius=0.7, fill="yellow")
image_sequence = [("Sun", sun)]
```

Copy the code into student_images.py

Create a scene!!

1 MAKE IT!

```
background = "space"
from turtle_shapes import *
sun = circle(x=2, y=2, radius=0.7,
             color="orange", fill="yellow")
stars = random_colors(
    star, count=3, x=-4, y=1, dx=1.5,
    size=0.7, palette=["yellow", "white"]
)
planet = circle(x=-1, y=-2, radius=1,
                color="purple", fill="pink")
image_sequence = [
    ("Sun", sun),
    ("Stars", stars),
    ("Planet", planet),
]
```

2 ADD IT TO YOUR SEQUENCE!

```
# The scene code above already includes
 # image_sequence.
```

Copy the code into student_images.py

Rotating Square!



1 MAKE IT!

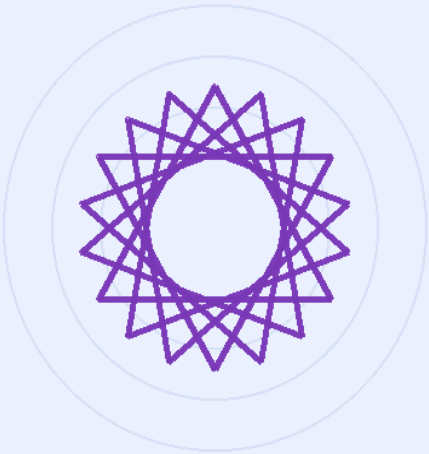
```
rotating_squares = []
for angle in range(0, 360, 30):
    rotating_squares += square(
        x=0, y=0, size=3,
        angle=angle, color="blue")
```

2 ADD IT TO YOUR SEQUENCE!

```
# Add rotating squares to your sequence
image_sequence = [
    ("Rotating Squares", rotating_squares),
]
```

Copy the code into student_images.py

Rotating Triangle!



1 MAKE IT!

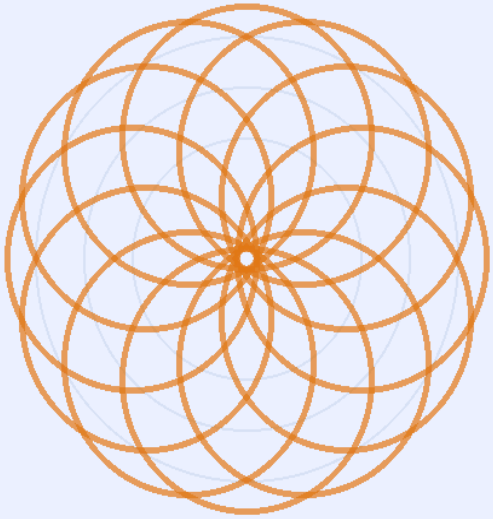
```
rotating_triangles = []
for angle in range(0, 360, 30):
    rotating_triangles += triangle(
        x=0, y=0, size=3,
        angle=angle, color="purple")
```

2 ADD IT TO YOUR SEQUENCE!

```
# Add rotating triangles to your sequence
image_sequence = [
    ("Rotating Triangles", rotating_triangles),
]
```

Copy the code into student_images.py

Rotating Circle!



1 MAKE IT!

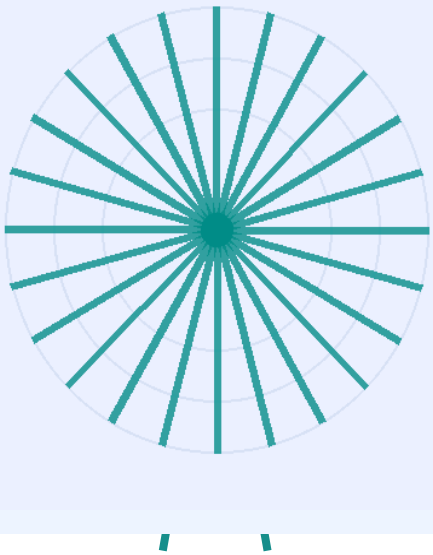
```
from math import cos, sin, radians
rotating_circles = []
for angle in range(0, 360, 45):
    a = radians(angle)
    rotating_circles += circle(
        x=2 * cos(a), y=2 * sin(a),
        radius=1, color="orange")
```

2 ADD IT TO YOUR SEQUENCE!

```
# Add rotating circles to your sequence
image_sequence = [
    ("Rotating Circles", rotating_circles),
]
```

Copy the code into student_images.py

Rotating Line!



1 MAKE IT!

```
rotating_lines = []
for angle in range(0, 360, 20):
    rotating_lines += [
        ("up", 0, 0),
        ("heading", angle),
        ("color", "teal"),
        ("width", 3),
        ("down",),
        ("forward", 3),
    ]
```

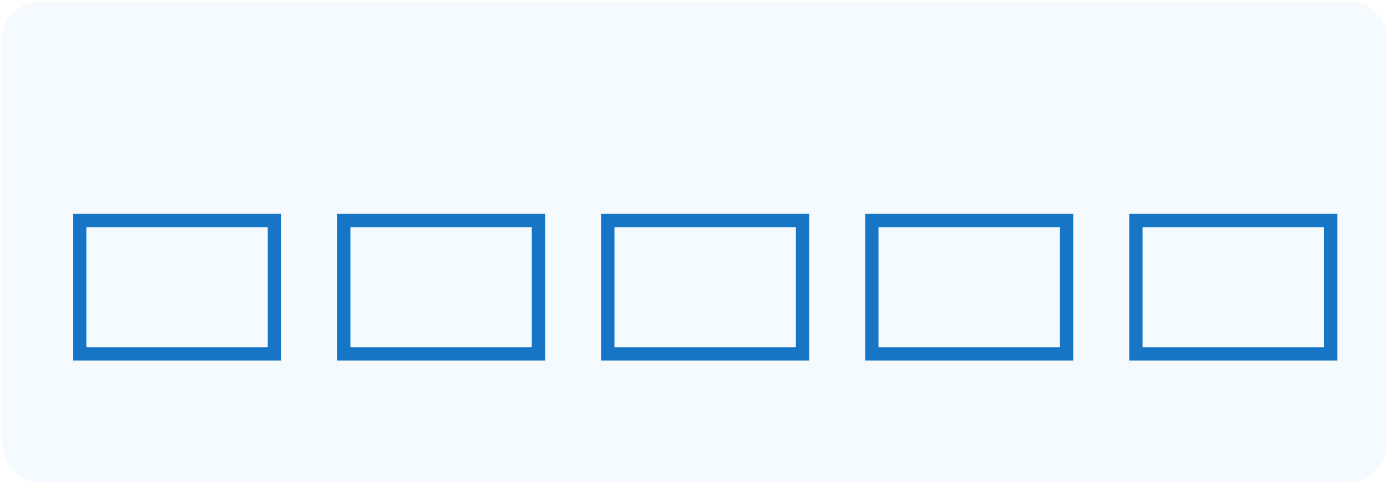
2 ADD IT TO YOUR SEQUENCE!

```
# Add rotating lines to your sequence
image_sequence = [
    ("Rotating Lines", rotating_lines),
]
```

Copy the code into student_images.py

Repeat a Shape!

Make 5 rectangles with one simple command.



1 MAKE IT!

```
# Repeat 5 blue rectangles
my_rectangles = repeat_shape(
    rectangle, count=5, x=-8, y=0,
    spacing=4, width=3, height=2,
    color="blue"
)
```

2 ADD IT TO YOUR SEQUENCE!

```
# Put this inside image_sequence = [ ... ]
("5 Rectangles", my_rectangles),
```

Try changing count=5 to count=3 or spacing=4 to spacing=2!

Repeat Your Own Drawing!

Make a line, then copy it again and again.



1. Make your drawing

```
my_line = [
    ("up", -5, -1),
    ("color", "blue"),
    ("width", 8),
    ("heading", 55),
    ("down",),
    ("forward", 2),
]
```

2. Repeat it 5 times

```
my_lines = repeat_shape(my_line, count=5, spacing=2)
```

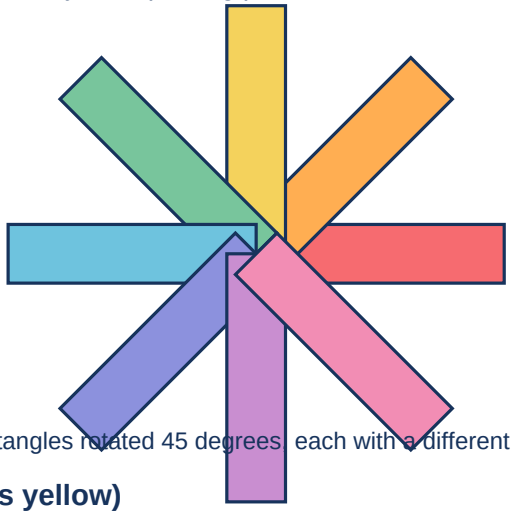
3. Add it to your sequence

```
# Add repeated lines to your sequence
("Repeated Lines", my_lines),
```

Try changing count or spacing to create your own pattern!

Rotate + Colour Repeated Shapes!

Use fill, random_fill, and palette to colour your repeating patterns.



Eight rectangles rotated 45 degrees, each with a different fill colour

1. One fill colour (all rectangles yellow)

```
yellow_shapes = repeat_shape(
    rectangle, count=8, rotate=45,
    width=3, height=1, fill="yellow"
)
```

2. Random fill colours (choose from a palette)

```
spinning_colours = repeat_shape(
    rectangle, count=8, x=0, y=0,
    dx=0, dy=0, rotate=45, width=3, height=1,
    random_fill=True, palette=["red", "blue", "yellow"]
)
```

3. Add BOTH drawings INSIDE image_sequence

```
image_sequence = [
    ("Yellow Shapes", yellow_shapes),
    ("Spinning Colours", spinning_colours),
]
```

What do the options mean?

color="blue" - sets the outline colour.

fill="yellow" - fills every shape with yellow.

random_fill=True - gives each repeated shape a random fill.

random_fill=False - switches random filling off.

palette=["red", "blue"] - limits random fills to those colours.

rotate=45 - turns each new copy another 45 degrees.

Tip: Keep the drawing definition above image_sequence, as in student_images.py.

Repeat Shapes Vertically!

Set dx=0 and use dy to move each copy up or down.



1. Yellow circles going up

```
vertical_circles = repeat_shape(
    circle, count=5, x=-5, y=-8,
    dx=0, dy=3, radius=1, fill="yellow"
)
```

2. Blue rectangles going up

```
vertical_rectangles = repeat_shape(
    rectangle, count=4, x=3, y=-7,
    dx=0, dy=4, width=3, height=2,
    fill="lightblue"
)
```

3. Put the drawings INSIDE image_sequence

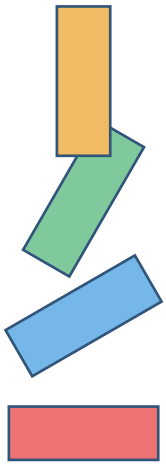
```
image_sequence = [
    ("Vertical Circles", vertical_circles),
    ("Vertical Rectangles", vertical_rectangles),
]
```

Vertical Stars and Spinning Colours!

Combine vertical movement with rotation and fill colours.



Rising Stars



Spinning Colours

1. Yellow stars that rise and turn

```
rising_stars = repeat_shape(
    star, count=4, x=-5, y=-6,
    dx=0, dy=3, rotate=20,
    size=2, fill="yellow"
)
```

2. Rectangles that rise, turn and change colour

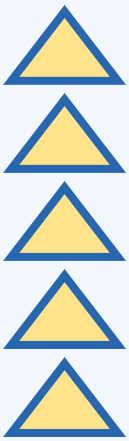
```
spinning_colours = repeat_shape(
    rectangle, count=4, x=3, y=-6,
    dx=0, dy=3, rotate=30,
    width=3, height=1, random_fill=True,
    palette=["red", "blue", "green", "orange"]
)
```

3. Put BOTH drawings INSIDE image_sequence

```
image_sequence = [
    ("Rising Stars", rising_stars),
    ("Spinning Colours", spinning_colours),
]
```

Vertical Triangles!

Repeat yellow triangles upwards and rotate each copy.



1. Create the pattern

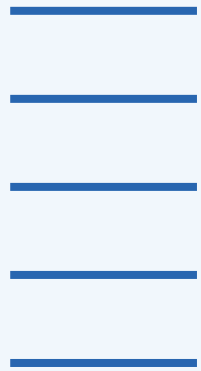
```
vertical_triangles = repeat_shape(
    triangle, count=5, x=-4, y=-8,
    dx=0, dy=3, size=2,
    rotate=15, fill="yellow"
)
```

```
2. Put the drawing INSIDE image_sequence:
image_sequence = [
    ("Vertical Triangles", vertical_triangles),
]
```

dx=0 keeps the drawing in one column. dy=3 moves each copy upwards.

Vertical Lines!

Create one line, then repeat it upwards.



1. Create the pattern

```
one_line = [
    ("up", 0, 0), ("heading", 0),
    ("down",), ("color", "blue"),
    ("width", 4), ("forward", 3),
]
```

```
vertical_lines = repeat_shape(
    one_line, count=5, x=-2, y=-6,
    dx=0, dy=3
)
```

```
image_sequence = [
    ("Vertical Lines", vertical_lines),
]
```

dx=0 keeps the drawing in one column. dy=3 moves each copy upwards.

Press F5 to Run Your Drawings!

Open student_images.py in Thonny. This is the only file students need to run.

1. CREATE

Make your shapes

2. CHOOSE

Set image_sequence

3. RUN

Press F5

In student_images.py:

```
yellow_shapes = repeat_shape(
    triangle, count=4, dx=0, dy=3,
    fill="yellow", size=2
)

spinning_colours = repeat_shape(
    shape_rectangle, count=8, rotate=45,
    width=3, height=1, random_fill=True
)

image_sequence = [
    ("Yellow Shapes", yellow_shapes),
    ("Spinning Colours", spinning_colours),
]

if __name__ == "__main__":
    from student_sequence import play_sequence
    play_sequence(image_sequence, colors, background)
```

Keep the final three lines at the bottom of student_images.py.

The player handles the drawing window, sound playlist and background.

Do not put the F5 runner inside the image_sequence brackets.

Make Your Own Breakbeat!

Use the WAV samples in your **sounds** folder to compose music with Python. Open **program_files/student_sounds.py** in Thonny.

1. Write a musical score

Each row contains 16 steps (one bar of four beats). **X** plays a sound; **-** is a rest. Spaces help you count the four beats.

```
tempo_bpm = 132

breakbeat_score = [
    {
        "kick":   "X--- --X- X--- X---",
        "snare":  "---- X--- ---- X---",
        "hihat":  "X-X- X-X- X-X- X-XX",
        "clap":   "---- X--- ---- X---",
        "bass":   "X--- ---- X--- --X-",
    },
    {
        "kick":   "X--- ---X X-X- ----",
        "snare":  "---- X--- ---- X--X",
        "hihat":  "X-XX X-X- X-XX XXXX",
        "clap":   "---- X--- ---- X---",
        "bass":   "X--- --X- X--- ----",
    },
]
```

2. Tell the app to play the score

```
sound_sequence = [
    ("score_breakbeat", tempo_bpm),
]
```

Save your work. Open **student_images.py** and press **F5**. The score is made into a looping beat while the Turtle draws.

Important: `score_breakbeat` is a special instruction, **not** a file called `breakbeat.wav`. It works with the updated `score_player.py` and `sound_effects.py`.

Experiment with Sound!

Change the tempo

BPM means **beats per minute**. Larger numbers make the beat faster. The current player accepts values from **60 to 960 BPM**.

```
tempo_bpm = 174  # Try 90, 132, 174 or 200

sound_sequence = [
    ("score_breakbeat", tempo_bpm),
]
```

Change the rhythm

Change the X and - symbols to create your own music. Keep **exactly 16 steps** per row, not counting spaces.

```
"kick":  "X--- X--- X--- X---"  # steady kicks
"snare": "---- X--- ---- X---"  # backbeat
"hihat": "XXXX XXXX XXXX XXXX"  # busy hats
```

Choose sounds from the library

A score row name must match a WAV filename in the **sounds** folder. For example, `kick` uses `sounds/kick.wav`. Use only sample names that actually exist in your project.

Your music checklist

1. Put your score in **program_files/student_sounds.py**.
2. Keep `sound_sequence = [("score_breakbeat", tempo_bpm)]`.
3. Check that **score_player.py** and the updated **sound_effects.py** are in **program_files**.
4. Keep the original sample WAV files in **sounds**.
5. Save, open **student_images.py**, and press **F5**.

If you get an error

Missing WAV for score_breakbeat: Your player is still the older version and needs the coded-score update.
Missing sample: kick.wav: Check the actual names in your sounds folder.
Sound format error: The score player expects compatible mono 16-bit WAV samples with the same sample rate.

Challenge: Make a quiet first bar and a busy second bar. Try 174 BPM for a drum-and-bass feel!

Make a Breakbeat with Chords

Grade 6 Turtle Music | student_sounds.py

Use drums, bass and chord samples in the SAME 16-step score.

X = play the sample. - = rest. Each bar contains 16 steps.

```
tempo_bpm = 132
breakbeat_score = [
    {
        "kick":     "X--- --X- X--- X---",
        "snare":    "---- X--- ---- X---",
        "hihat":    "X-X- X-X- X-X- X-XX",
        "bass":     "X--- ---- X--- --X-",
        "chord_C":  "X--- ---- ---- ----",
        "chord_G":  "---- ---- X--- ----",
    },
    {
        "kick":     "X--- ---X X-X- ----",
        "snare":    "---- X--- ---- X--X",
        "hihat":    "X-XX X-X- X-XX XXXX",
        "chord_Am": "X--- ---- ---- ----",
        "chord_F":  "---- ---- X--- ----",
    },
]
```

Tell the sound player to use the score:

```
sound_sequence = [("score_breakbeat", tempo_bpm)]
```

The chord names must match WAV files in your sounds folder.

Examples included: chord_C, chord_G, chord_Am, chord_F.

Open student_images.py and press F5 to play the music and draw.

Experiment with Your Music

Mix rhythms, melodies and sustained chords

Change the tempo

```
tempo_bpm = 174
sound_sequence = [("score_breakbeat", tempo_bpm)]
```

Try a chord on the final step

```
"chord_C": "---- ---- ---- ---X"
```

The chord continues across the next bar or the loop boundary.

The score player overlaps WAV sounds rather than cutting at each bar.

Add another library sound

```
"shaker": "X-X- X-X- X-X- X-X-",
"chord_Dm": "X--- ---- ---- ----"
```

Each row needs exactly 16 X or - characters (spaces are ignored).

Use only names of WAV files already inside sounds/.

Files needed in program_files/

student_sounds.py - edit the score

score_player.py - combines the samples

sound_effects.py - starts and loops the music

No new sound files are needed. The score is generated when you run.